Insertion Sort — Analysis and Reflection

Insertion Sort builds a sorted subarray one element at a time, shifting each new element leftward until it reaches its correct position. It runs in O(n) time in the best case, O(n2) in the average and worst cases, and uses O(1) auxiliary space, making it in-place and stable.

Best Case. On an already-sorted array of 500 elements the measured time was 0.09 ms. The inner loop never executes, so each element is placed in one comparison, yielding linear O(n) behaviour. This makes Insertion Sort faster than divide-and-conquer algorithms for nearly-sorted data since those carry fixed recursion overhead regardless of order.

Worst Case. A reversed 500-element array took 8.07 ms. Every element must shift past all its predecessors, producing n(n-1)/2 comparisons and O(n2) growth. Times scale quadratically, making the algorithm impractical for large unsorted datasets.

Average Case. Random arrays of increasing size averaged 0.21 ms at n=100, 1.14 ms at n=200, 1.41 ms at n=300, 2.68 ms at n=400, and 4.42 ms at n=500. This confirms quadratic growth in practice, with roughly half the constant of the worst case.

Space Complexity. The algorithm is in-place, requiring only a single temporary variable regardless of input size — O(1) auxiliary space. This is advantageous in memory-constrained settings compared to Merge Sort's O(n) or Quick Sort's O(log n) stack usage.

Stability. Sorting [(3,alice),(1,bob),(2,carol),(1,dave),(2,eve)] by the first element yields [(1,bob),(1,dave),(2,carol),(2,eve),(3,alice)]. Equal keys retain their original relative order because the inner loop uses strict greater-than and never swaps equal elements past each other.

Efficiency Discussion. For small arrays (roughly under 50 elements) Insertion Sort's low overhead and sequential memory access make it competitive with or faster than O(n log n) algorithms. For large arrays O(n2) dominates and Quick Sort or Merge Sort are preferable. Bubble Sort shares the same asymptotic class but performs more swaps; Quick Sort is faster on average but unstable and O(n2) in its worst case.

Practical Applications. Insertion Sort is well suited to nearly-sorted data streams, embedded systems with strict memory limits, and maintaining a sorted list after incremental inserts. Its stability is useful whenever preserving secondary ordering matters, and its simplicity makes it a common teaching example.

Improvements and Variations. Binary Insertion Sort uses binary search to find the insertion point in O(log n) comparisons, reducing comparison count though shifting still costs O(n2). Shell Sort generalises the approach with gap sequences, reaching roughly O(n1.5). Most practically, TimSort — Python's built-in sort — uses Insertion Sort on small natural runs before merging them, combining its best-case linear strength with O(n log n) overall performance.

Unit Tests. Three regular cases: [3,1,2] sorts to [1,2,3]; [1,2,3] remains [1,2,3]; [-3,-1,-2] sorts to [-3,-2,-1]. Three edge cases: an empty list returns an empty list; a single-element list [42] is unchanged; duplicates [2,2,1,1] sort correctly to [1,1,2,2]. All six tests pass.